EcoRisk Nexus - AI-Driven Risk and Sustainability Analysis

MVP Final Version

1. Background:

While ESG (Environmental, Social, and Governance) reporting has become increasingly important worldwide, the United States lags Europe in terms of mandatory disclosure regulations. However, this landscape is set to change. By 2026, California is expected to enforce its first mandatory ESG reporting law, including significant greenhouse gas (GHG) emissions disclosure requirement (Ropes & Gray, 2024.).

One critical focus area in ESG reporting is **CO2 emissions**, particularly Scope 3 emissions, which encompass indirect emissions from upstream and downstream activities in a company's value chain (U.S. Environmental Protection Agency [EPA], 2024.). These emissions are challenging to measure yet vital for comprehensive environmental impact assessment.

2. Problem Statement

Despite the growing need for ESG compliance, U.S.-based SMEs face significant barriers:

1. A lack of technical expertise and tools to collect and analyze sustainability data effectively.
2. Challenges in forecasting CO2 emissions, particularly Scope 3 emissions, due to complex data requirements and a lack of accessible tools.
3. Insufficient preparation for impending regulations, risking operational disruptions and reputational damage when laws take effect.

Addressing these challenges requires a low-cost, user-friendly solution tailored to the unique needs of SMEs, allowing them to integrate sustainability practices seamlessly into their operations.

3. Solution Imperative

To avoid chaos when mandatory ESG reporting laws come into force, companies must proactively adopt tools and processes for data collection and analysis. By implementing solutions like **EcoRisk Nexus**, organizations can stay ahead of regulations and position themselves as sustainability leaders.

4. Technical Documentation

I. Technical Workflow

The **EcoRisk Nexus MVP** leverages Low-code AI principles to streamline sustainability and CO2 emissions analysis into a three-step workflow:

1. **Data Preparation**:
   - Users upload raw datasets containing key operational and environmental metrics, including fields such as Energy Consumption, CO2 Emissions, and Waste Generation.
   - Interaction features such as Energy Downtime Ratio, Incident Waste Product, and CO2 Energy Ratio are calculated automatically, removing the need for manual preprocessing.
   - Validation datasets are used to test predictions, simulating real-world scenarios where certain metrics like Sustainability Scores are initially unavailable.
2. **Model Implementation**:
   - The platform integrates two AI models to generate predictions:
     - **Sustainability Score Prediction**: A Gradient Boosting Model (XGBoost) predicts sustainability performance based on core and derived metrics.
     - **CO2 Emission Forecasting**: SARIMA and ARIMA models provide six-month forward-looking CO2 emissions forecasts, tailored to the company or Project-wise historical data availability.
   - The models are designed to handle varying data sizes, adapting flexibly to ensure accurate predictions even with limited historical data.
3. **Integration and Output**:
   - Predictions from both models are merged into a single dataset, ensuring that Sustainability Scores and CO2 forecasts are readily accessible in one comprehensive file.
   - The final output can be downloaded as CSV file, designed compatible with Tableau and other data visualization tools.
   - This approach makes advanced AI tools accessible to SMEs without requiring technical expertise, aligning with the Low-code AI philosophy.

II. Data Source

1. **Training Data:**
   a. The training dataset for the Sustainability Score prediction model was synthetically generated to simulate operational and environmental metrics for SMEs. Baseline variables were based on publicly available references:
      i. **Incident Frequency**: U.S. Bureau of Labor Statistics (2020)
      ii. **Downtime Hours**: U.S. Energy Information Administration (2022)
      iii. **Energy Consumption**: U.S. Energy Information Administration (n.d.)
      iv. **CO2 Emissions**: U.S. Department of Energy Industrial Sector (2018)
      v. **Waste Generation**: Environmental Protection Agency (n.d.)
      vi. **Renewable Energy Usage**: U.S. Energy Information Administration (2021)
      vii. **Water Usage**: United States Geological Survey (n.d.)
      viii. **Recycling Rate**: U.S. Environmental Protection Agency (n.d.)
   b. Interaction features were derived to improve the model's predictive power:
      i. **Energy Downtime Ratio** = Energy Consumption / Downtime Hours

ii. **Incident Waste Product** = Incident Frequency × Waste Generation
iii. **CO2 Energy Ratio** = CO2 Emissions / Energy Consumption

2. **Validation Data for Sustainability Score, Training Data for CO2 Emiossions prediction:**
Validation dataset was created with random entries, simulating companies and project-level data across specific dates. This dataset was used for two purposes:
   a. Sustainability Score Validation: The dataset excluded the Sustainability Score to test model predictions for the sustainability scoring model.
   b. CO2 Emissions Model Training: The same dataset served as the training data for the CO2 emissions prediction model.

   It included:
   - Core features (e.g., Energy Consumption, CO2 Emissions, Waste Generation)
   - Identifiers: Company Name, Project ID, and Date

III. Modeling Approach

1. **Sustainability Score Prediction:**
   a. **Model Type**: Gradient Boosting Model (XGBoost)
   b. **Features**: Core and derived features such as Incident Frequency, Downtime Hours, Energy Downtime Ratio, and CO2 Energy Ratio.
   c. **Training Process**:
      - The data was split into training and test sets (80%-20% split).
      - The XGBoost model was trained with default parameters, optimized for regression tasks.
      - Feature importance analysis highlighted the most impactful variables for sustainability predictions.
   d. **Performance Metrics**:
      - Mean Squared Error (MSE): 71.56
      - R^2 Score: 0.7521
   e. **Output**: A trained XGBoost model saved as sustainability_score_xg_model.pkl.

2. **CO2 Emission Forecasting:**
   a. **Model Type**: SARIMA/ARIMA
   b. **Features**: Historical CO2 emissions aggregated at the company level.
   c. **Process**:
      i. Data was grouped by Company and Date, with missing values handled through interpolation.
      ii. SARIMA was applied for companies with 12 or more monthly data points. ARIMA was used for smaller datasets due to limited seasonal information.
      iii. Predictions provided six months of CO2 emissions forecasts.
   d. **Output**:
      i. A CSV file, company_wise_co2_forecasts.csv, contains forecasted CO2 emissions for each company along with confidence intervals.

3. **Integrated Model Value:**
By combining XGBoost and SARIMA/ARIMA models, the MVP offers a robust solution for both sustainability assessment and emissions forecasting. This dual-model approach ensures that the system delivers comprehensive insights tailored to each company's operational and environmental data.

IV. Integration with Streamlit: Prediction and File Creation

The deployment of EcoRisk Nexus via Streamlit ensures a seamless user experience, transforming Python-based AI models into an accessible web application. Key features of this integration include:

1. **Streamlined User Workflow**:
    - **File Upload**: Users can drag and drop their dataset into the application or browse their files.
    - **Automated Analysis**: Once the file is uploaded, the app calculates interaction features, applies AI models, and merges predictions into a unified dataset without requiring manual intervention.
    - **Prediction Outputs**: Results are presented directly within the app and are also available for download in CSV format.
2. **User-Centric Design**:
    - The application interface is designed with simplicity in mind, ensuring that non-technical users can interact with the platform effortlessly.
    - Clear instructions, visual feedback, and downloadable results minimize the learning curve.
3. **Scalable AI Deployment**:
    - Streamlit acts as a Low-code platform that bridges complex AI models with practical end-user functionality.
    - By abstracting the backend processes, users interact with the app through a simple interface, while the heavy computation is handled in the background.
4. **Future-Ready Design**:
    - The app is compatible with major visualization tools such as Tableau, Power BI, and Amazon QuickSight, ensuring seamless integration for advanced analysis. These tools also offer mobile-friendly design, enabling users to access predictions conveniently on any device.
    - The flexibility of Streamlit enables rapid iteration, allowing enhancements based on user feedback and evolving requirements.

5. Testing and Refinement

I. **Testing Process**

- The MVP was tested iteratively with sample datasets to evaluate model performance and user interface functionality.
- Key improvements included handling missing data, optimizing prediction algorithms, and refining the user interface for better usability.

II. **Feedback Insights**

- **Feedback from teams:**
  The app received positive feedback from team members for its ease of use and the comprehensive output it generates. However, several recommendations were made to enhance its functionality further:
  - Automated Integration with Visualization Tools: Teams suggested incorporating automated data integration with platforms like Tableau, Power BI, or Amazon QuickSight to streamline the visualization process.
  - Incorporating Additional Predictive Factors: Expanding the range of predictive capabilities to include other sustainability-related metrics was highlighted as a potential enhancement.
  - Improving Prediction Accuracy: Teams emphasized the importance of refining the CO2 emissions forecasting model to achieve greater precision, particularly for smaller datasets.
- **Executive input at presentation:**
  Executives provided insightful feedback regarding the app's relevance and potential impact on SMEs (Small and Medium Enterprises):
  - **Challenges in Data Collection** It was noted that SMEs often face significant difficulties in gathering accurate and comprehensive sustainability-related data. While this challenge is not directly related to the app itself, it could impact its adoption if users are unable to provide the necessary data for meaningful predictions.
  - **Coding Expertise and Outsourcing:**
    In organizations lacking technical expertise, coding tasks are frequently outsourced. However, this approach is often associated with challenges such as extended timelines, misunderstandings of requirements, and higher costs. Therefore, a model that integrates the entire process, from capturing customer requirements to building the system, is considered an ideal solution.
  - **Simplified and Flexible Design**: Executive praised the app's simplicity, flexibility, and integration-ready design, noting that these characteristics make it highly appealing for SMEs with limited resources and technical expertise.

6. Conclusion and Next Steps

I. Conclusion

Our sustainability assessment solution, **EcoRisk Nexus**, provides SMEs with a robust platform for predicting sustainability scores and CO2 emissions. Through a Low-code AI approach, the MVP has demonstrated its potential to bridge the gap between complex sustainability challenges and user-friendly solutions. By focusing on simplicity, flexibility, and scalability, we have laid the groundwork for a platform that aligns with evolving regulatory landscapes and user needs.

II. Next Steps

To ensure effective development and scalability, we have structured our roadmap into short-term, mid-term, and long-term goals:

1. **Short-Term Goals**:
    - Complete MVP development using MySQL and Python for efficient data processing.
    - Build initial dashboards in Tableau, Power BI, and Amazon QuickSight to gather early feedback and validate the platform's core functionalities.
2. **Mid-Term Goals**:
    - Integrate advanced predictive analytics into the platform, enabling smarter sustainability insights.
    - Partner with SMEs to conduct testing and validation, ensuring the solution meets industry standards and user expectations.
3. **Long-Term Goals**:
    - Secure necessary certifications to establish credibility and align with regulatory standards.
    - Expand the platform's functionalities to cater to broader industries, positioning EcoRisk Nexus as a trusted sustainability assessment tool across various sectors.

III. Call to Action

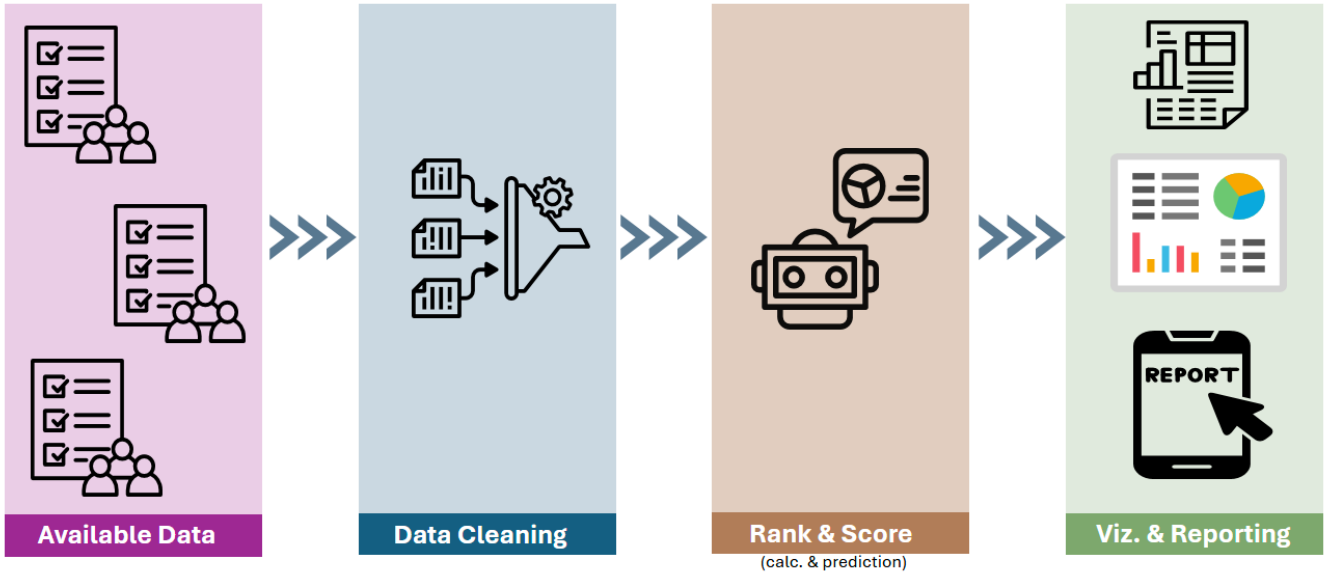
We invite potential partners and investors to collaborate with us in shaping the future of sustainability assessment. By engaging with us early, you can actively contribute to the product's development and gain a competitive advantage in the market as sustainability regulations become mandatory. Together, we can unlock the full potential of EcoRisk Nexus and drive sustainable growth across industries.

7. References

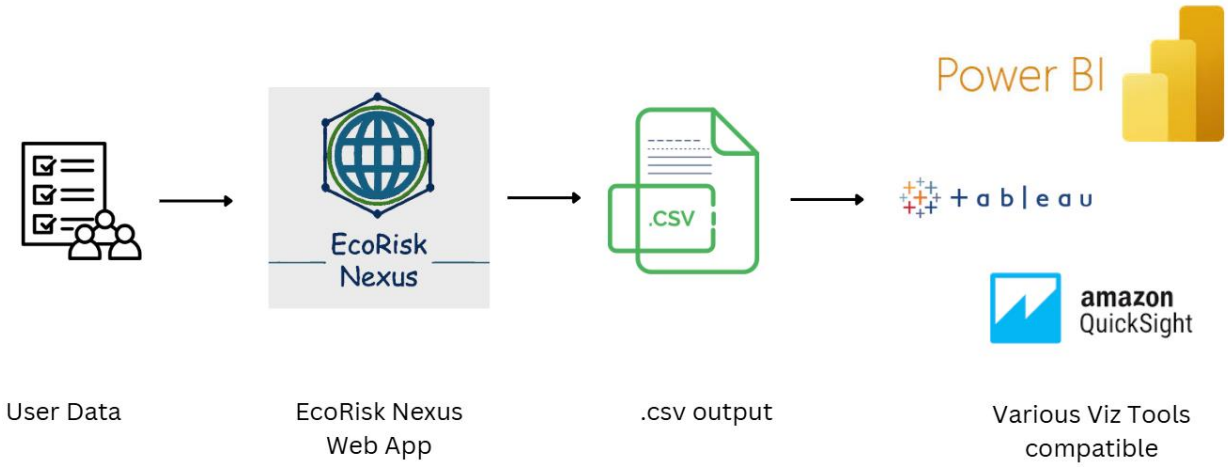
1. Ropes & Gray. (n.d.). California GHG emissions disclosure enforcement guidance. https://www.ropesgray.com/en/insights/viewpoints/102jr1k/california-issues-important-enforcement-guidance-for-its-new-ghg-emissions-disclo
2. EIA. (n.d.). Manufacturing energy survey. https://www.eia.gov/consumption/manufacturing/
3. Environmental Protection Agency. (n.d.). EPA industrial waste estimates. https://www.epa.gov/ghgemissions/global-greenhouse-gas-emissions-data
4. EPA Climate Leadership Scope 3 Inventory Guidance. EPA Website. Retrieved from https://www.epa.gov/climateleadership/scope-3-inventory-guidance#:~:text=Scope%203%20emissions%20are%20the,its%20upstream%20and%20downstream%20activities
5. Google. (2024.). *Colaboratory*. (December 17th version) https://colab.research.google.com
6. Google DeepMind. (2024). Gemini (December 17th version) [Large language model]. https://www.deepmind.com
7. OpenAI. (2024). *ChatGPT* (December 17th version) [Large language model]. https://openai.com
8. Statsmodels SARIMA documentation. (n.d.). https://www.statsmodels.org/stable/
9. Streamlit documentation. (n.d.). https://docs.streamlit.io/
10. U.S. Bureau of Labor Statistics. (2020). U.S. Bureau of Labor Statistics annual report. https://www.bls.gov/
11. U.S. Department of Commerce. (2024). Proposed sustainability mandates for SMEs in 2026. https://www.commerce.gov
12. U.S. Department of Energy Industrial Sector. (2018). Annual industrial report. https://www.energy.gov/eere/amo/annual-industrial-sector-report
13. U.S. Energy Information Administration. (2021). National energy average. https://www.eia.gov/
14. U.S. Energy Information Administration. (2022). U.S. Energy outlook report. https://www.eia.gov/outlooks/
15. United States Geological Survey. (n.d.). Industrial water usage report. https://www.usgs.gov/
16. U.S. Environmental Protection Agency. (n.d.). National waste recycling average. https://www.epa.gov/
17. XGBoost documentation. (n.d.). https://xgboost.readthedocs.io/

8. Appendices

Appendix 1: MVP IDEA



Appendix 2: EcoRisk Nexus: From Prediction to Visualization



Appendix 3: Data Dictionary

Appendix Table-1: Baseline Indicators for Random Dataset Generation

Indicator	Baseline Max	Baseline Min	Unit	Source
Incident_Frequency	2.7	0	Incidents/month	U.S. Bureau of Labor Statistics (2020) [10]
Downtime_Hours	5.5	0	Hours/month	U.S. Energy Information Administration (2022)[12]
Energy_Consumption	2000	100	kWh/month	EIA Manufacturing Energy Survey [2]
CO2 Emissions	2000	100	kg/month	U.S. DOE Industrial Sector (2018) [10]
Waste_Generation	500	50	kg/month	EPA Industrial Waste Estimates [14]
Renewable_Energy_Usage	12	5	% of energy used	U.S. EIA (2021 National Average) [11]
Water_Usage	2000	500	Liters/month	USGS Industrial Water Usage [13]
Recycling_Rate	100	35	% of waste recycled	U.S. EPA National Average [14]
Safety_Compliance_Score	100	0	Score (0-100)	Self-defined

Appendix 4: Random Dataset Creation Python Code

Randam Dataset Creation Python code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set random seed for reproducibility
np.random.seed(42)

# Number of rows for the dataset
num_rows = 2000

# Generate base features adhering to baseline
ranges
Incident_Frequency = np.random.uniform(0, 2.7,
num_rows)  # 0 - 2.7 Incidents/month
Downtime_Hours = np.random.uniform(0, 5.5,
num_rows)  # 0 - 5.5 Hours/month
Energy_Consumption = np.random.uniform(100, 2000,
num_rows)  # 100 - 2000 kWh/month
CO2_Emissions = np.random.uniform(100, 2000,
num_rows)  # 100 - 2000 kg/month
Waste_Generation = np.random.uniform(50, 500,
num_rows)  # 50 - 500 kg/month
Renewable_Energy_Usage = np.random.uniform(5, 12,
num_rows)  # 5% - 12% of energy used
Water_Usage = np.random.uniform(500, 2000,
num_rows)  # 500 - 2000 Liters/month
Recycling_Rate = np.random.uniform(35, 100,
num_rows)  # 35% - 100% of waste recycled

# Step 1: Generate Sustainability_Score with 3
distributions
Sustainability_Score = np.zeros(num_rows)

# High scores: 70-100 (80%)
high_score_indices =
np.random.choice(range(num_rows),
size=int(num_rows * 0.8), replace=False)
Sustainability_Score[high_score_indices] =
np.random.uniform(70, 100,
len(high_score_indices))

# Medium scores: 40-70 (15%)
remaining_indices = np.setdiff1d(range(num_rows),
high_score_indices)
medium_score_indices =
np.random.choice(remaining_indices,
size=int(num_rows * 0.15), replace=False)
Sustainability_Score[medium_score_indices] =
np.random.uniform(40, 70,
len(medium_score_indices))

# Low scores: 30-40 (5%)
low_score_indices =
np.setdiff1d(remaining_indices,
medium_score_indices)
Sustainability_Score[low_score_indices] =
np.random.uniform(30, 40, len(low_score_indices))

# Adjust other features for different score
ranges
Recycling_Rate[high_score_indices] = 100
Incident_Frequency[high_score_indices] =
np.random.uniform(0, 0.5,
len(high_score_indices))
Downtime_Hours[high_score_indices] =
np.random.uniform(0, 1, len(high_score_indices))

Recycling_Rate[medium_score_indices] =
np.random.uniform(50, 70,
len(medium_score_indices))
Incident_Frequency[medium_score_indices] =
np.random.uniform(1, 2,
len(medium_score_indices))
Downtime_Hours[medium_score_indices] =
np.random.uniform(2, 4,
len(medium_score_indices))

Recycling_Rate[low_score_indices] = 35



Incident_Frequency[low_score_indices] =
np.random.uniform(2, 2.7, len(low_score_indices))
Downtime_Hours[low_score_indices] =
np.random.uniform(4.5, 5.5,
len(low_score_indices))

# Create DataFrame
df = pd.DataFrame({
    'Incident_Frequency': Incident_Frequency,
    'Downtime_Hours': Downtime_Hours,
    'Energy_Consumption': Energy_Consumption,
    'CO2_Emissions': CO2_Emissions,
    'Waste_Generation': Waste_Generation,
    'Renewable_Energy_Usage':
Renewable_Energy_Usage,
    'Water_Usage': Water_Usage,
    'Recycling_Rate': Recycling_Rate,
    'Sustainability_Score': Sustainability_Score
})

# Save the dataset to CSV
df.to_csv('sustainability_dataset.csv',
index=False)
print("Dataset created and saved as
'sustainability_dataset.csv'")

# Plot the distribution
plt.figure(figsize=(10, 6))
sns.histplot(Sustainability_Score, bins=30,
kde=True, color="skyblue")
plt.title("Sustainability_Score Distribution (70-
100: 80%, 40-70: 15%, Below 40: 5%)")
plt.xlabel("Sustainability_Score")
plt.ylabel("Frequency")
plt.show()
```

Validation Random Dataset Creation Python code

```
import pandas as pd
import numpy as np
import random
from datetime import datetime, timedelta

# Set random seed for reproducibility
np.random.seed(42)
random.seed(42)
```

```
# Number of rows
num_rows = 2000

# Generate base features
Incident_Frequency = np.random.uniform(0, 2.7,
num_rows)
Downtime_Hours = np.random.uniform(0, 5.5,
num_rows)
Energy_Consumption = np.random.uniform(100, 2000,
num_rows)
CO2_Emissions = np.random.uniform(100, 2000,
num_rows)
Waste_Generation = np.random.uniform(50, 500,
num_rows)
Renewable_Energy_Usage = np.random.uniform(5, 12,
num_rows)
Water_Usage = np.random.uniform(500, 2000,
num_rows)
Recycling_Rate = np.random.uniform(35, 100,
num_rows)

# Generate Company Names
company_names = [f"company_{i}" for i in range(1,
11)]
Company = random.choices(company_names,
k=num_rows)

# Generate Project_IDs
Project_ID = [f"s_{i+1}" for i in
range(num_rows)]

# Generate random Dates within a year (2023)
start_date = datetime(2023, 1, 1)
Date = [(start_date +
timedelta(days=random.randint(0,
364))).strftime('%Y-%m-%d') for _ in
range(num_rows)]

# Create DataFrame without targets
validation_data = pd.DataFrame({
    'Company': Company,
    'Project_ID': Project_ID,
    'Date': Date,
    'Incident_Frequency': Incident_Frequency,
    'Downtime_Hours': Downtime_Hours,
    'Energy_Consumption': Energy_Consumption,
    'CO2_Emissions': CO2_Emissions,
    'Waste_Generation': Waste_Generation,
    'Renewable_Energy_Usage':
Renewable_Energy_Usage,
    'Water_Usage': Water_Usage,
    'Recycling_Rate': Recycling_Rate
})

# Save the updated dataset without targets
validation_data.to_csv('validation_dataset.csv',
index=False)
print("Validation dataset without targets saved
as 'validation_dataset.csv'.")
```

Appendix 5: Python Code for Sustainability Score prediction

Model Creation and Saving

```
from lightgbm import LGBMRegressor
from sklearn.model_selection import
train_test_split
from sklearn.metrics import mean_squared_error,
r2_score
import joblib
import pandas as pd

# Load the dataset
data =
pd.read_csv('sustainability_dataset_with_features
.csv')

# Define features and target
X = data.drop(columns=['Sustainability_Score'])
# Features
y = data['Sustainability_Score']                #
Target

# Split the dataset
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)

# Initialize LightGBM Regressor
model = LGBMRegressor(n_estimators=200,
learning_rate=0.05, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Predict on the test set
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-squared (R²): {r2:.4f}")

# Save the trained model
joblib.dump(model,
'sustainability_score_model.pkl')
print("\nModel saved as
'sustainability_score_model.pkl'.")

--------------
XGBoost       - MSE: 71.56, R²: 0.7521
```

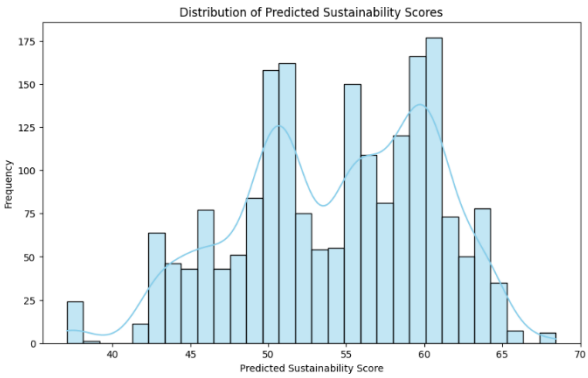
Model Demonstration

```
import pandas as pd
import joblib

# Load the validation dataset
validation_data =
pd.read_csv('validation_dataset.csv')

# Add derived features for prediction
```

```
validation_data['Energy_Downtime_Ratio'] =
validation_data['Energy_Consumption'] /
(validation_data['Downtime_Hours'] + 1e-5)
validation_data['Incident_Waste_Product'] =
validation_data['Incident_Frequency'] *
validation_data['Waste_Generation']
validation_data['CO2_Energy_Ratio'] =
validation_data['CO2_Emissions'] /
(validation_data['Energy_Consumption'] + 1e-5)

# Load the trained model
model =
joblib.load('sustainability_score_model.pkl')

# Define features for prediction
features = ['Incident_Frequency',
'Downtime_Hours', 'Energy_Consumption',
            'CO2_Emissions', 'Waste_Generation',
'Renewable_Energy_Usage',
            'Water_Usage', 'Recycling_Rate',
'Energy_Downtime_Ratio',
            'Incident_Waste_Product',
'CO2_Energy_Ratio']

# Predict Sustainability Scores
validation_data['Predicted_Sustainability_Score']
= model.predict(validation_data[features])

# Save the results
validation_data.to_csv('validation_with_predictio
ns.csv', index=False)
print("\nPredictions saved successfully!")
```

Visualization for Validation

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Load the dataset with predictions
data =
pd.read_csv('validation_with_predictions.csv')

# Plot the distribution of
Predicted_Sustainability_Score
plt.figure(figsize=(10, 6))
sns.histplot(data['Predicted_Sustainability_Score
'], bins=30, kde=True, color="skyblue")

# Add titles and labels
plt.title("Distribution of Predicted
Sustainability Scores")
plt.xlabel("Predicted Sustainability Score")
plt.ylabel("Frequency")

# Show the plot
plt.show()
```



Appendix Figure-1: Distribution of Predicted Sustainability Scores

Appendix 6: Python Code for Sustainability Score prediction

Model Creation and Saving

```
from statsmodels.tsa.statespace.sarimax import
SARIMAX
import pandas as pd

# Load dataset and group by Company and Date
data = pd.read_csv('validation_dataset.csv',
parse_dates=['Date'])
company_grouped_data = data.groupby(['Company',
'Date'])['CO2_Emissions'].sum().reset_index()

# Forecast function with SARIMA/ARIMA condition
def forecast_company_co2(data, company, steps=6):
    company_data = data[data['Company'] ==
company].set_index('Date')['CO2_Emissions']
    company_data = company_data.dropna()
    if not company_data.empty:
        company_data = company_data.asfreq('M')

    if company_data.empty:
        print(f"No data found for company:
{company}. Skipping...")
        return None, None, None

    last_date = company_data.index.max()
    try:
        if len(company_data) < 12:
            print(f"Using ARIMA for {company} due
to insufficient data.")
            seasonal_order = (0, 0, 0, 0)
        else:
            print(f"Using SARIMA for {company}
with sufficient data.")
            seasonal_order = (1, 1, 1, 12)

        model = SARIMAX(company_data, order=(1,
1, 1), seasonal_order=seasonal_order,
enforce_stationarity=False)
        results = model.fit(disp=False)
    except Exception as e:
        print(f"Failed to build model for
{company}: {e}")
        return None, None, None
```

```
        forecast = results.get_forecast(steps=steps)
        forecast_mean = forecast.predicted_mean
        forecast_conf_int = forecast.conf_int()

        forecast_dates =
    pd.date_range(start=last_date +
    pd.Timedelta(days=1), periods=steps, freq='M')
        return forecast_dates, forecast_mean,
    forecast_conf_int
```

Model Validation

```
    import pandas as pd

    # Forecast for all companies
    company_forecasts = []
    for company in
company_grouped_data['Company'].unique():
        forecast_dates, forecast_mean,
forecast_conf_int =
forecast_company_co2(company_grouped_data, company)
        if forecast_dates is not None:
            forecast_df = pd.DataFrame({
                'Date': forecast_dates,
                'Company': company,
                'Forecasted_CO2_Emissions':
forecast_mean.values,
                'Lower_CI': forecast_conf_int.iloc[:,
0].values,
                'Upper_CI': forecast_conf_int.iloc[:,
1].values
            })
            company_forecasts.append(forecast_df)

    # Combine all forecasts into a single DataFrame
    forecast_results = pd.concat(company_forecasts,
ignore_index=True)

    # Save forecast results
    forecast_results.to_csv('company_wise_co2_forecas
ts.csv', index=False)
    print("Saved forecast results to
'company_wise_co2_forecasts.csv'.")
```

Appendix 7: Python Code for Streamlit Integration

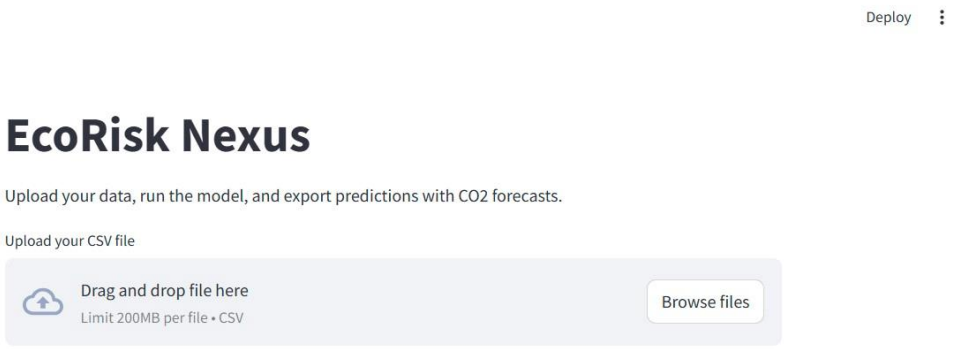
```
    import pandas as pd
    import numpy as np
    import joblib
    import streamlit as st
    from statsmodels.tsa.statespace.sarimax import
    SARIMAX

    # ---------------------------- Title ------------
    ----------------
    st.title("EcoRisk Nexus")
    st.write("Upload your data, run the model, and
    export predictions with CO2 forecasts.")

    # ---------------------------- Step 1: Upload
    Dataset ----------------------------
    uploaded_file = st.file_uploader("Upload your CSV
    file", type=["csv"])
if uploaded_file:
    # Load the uploaded dataset
    data = pd.read_csv(uploaded_file,
parse_dates=['Date'])
    st.write("Input Data Sample:")
    st.write(data.head())

    # ---------------------------- Step 2: Add
Derived Features ----------------------------
    st.write("Adding derived features for
Sustainability Score prediction...")

    # Add derived features
    data['Energy_Downtime_Ratio'] =
data['Energy_Consumption'] /
(data['Downtime_Hours'] + 1e-5)
    data['Incident_Waste_Product'] =
data['Incident_Frequency'] *
data['Waste_Generation']
    data['CO2_Energy_Ratio'] =
data['CO2_Emissions'] /
(data['Energy_Consumption'] + 1e-5)

    st.write("Derived features added
successfully!")
    st.write(data[['Energy_Downtime_Ratio',
'Incident_Waste_Product',
'CO2_Energy_Ratio']].head())

    # ---------------------------- Step 3: Load
Sustainability Score Model ----------------------
------
    st.write("Loading the pre-trained
Sustainability Score model...")
    model =
joblib.load('sustainability_score_xg_model.pkl')
    st.write("Model loaded successfully!")

    # ---------------------------- Step 4:
Predict Sustainability Score --------------------
--------
    # Define features for prediction
    features = ['Incident_Frequency',
'Downtime_Hours', 'Energy_Consumption',
                'CO2_Emissions',
'Waste_Generation', 'Renewable_Energy_Usage',
                'Water_Usage', 'Recycling_Rate',
'Energy_Downtime_Ratio',
                'Incident_Waste_Product',
'CO2_Energy_Ratio']

    # Ensure that all required features exist in
the data
    X = data[features]

    # Predict Sustainability Score
    st.write("Predicting Sustainability
Score...")
    data['Predicted_Sustainability_Score'] =
model.predict(X)

    # Display the results
    st.write("Prediction Results (Sustainability
Score):")
```

```
    st.write(data[['Company', 'Project_ID',
'Date',
'Predicted_Sustainability_Score']].head())

  # ---------------------------- Step 5: Forecast
CO2 Emissions ----------------------------
    st.write("Forecasting CO2 Emissions...")

    # Group data by Company and Date
    company_grouped_data =
data.groupby(['Company',
'Date'])['CO2_Emissions'].sum().reset_index()
    company_grouped_data['Date'] =
pd.to_datetime(company_grouped_data['Date'])
    company_grouped_data =
company_grouped_data.sort_values(by=['Company',
'Date']).reset_index(drop=True)

    # Define forecast function
    def forecast_company_co2(data, company,
steps=6):
        company_data = data[data['Company'] ==
company].set_index('Date')['CO2_Emissions']
        company_data = company_data.dropna()
        if not company_data.empty:
            company_data =
company_data.asfreq('M')
        if company_data.empty:
            return None
        try:
            model = SARIMAX(company_data,
order=(1, 1, 1), seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False)
            results = model.fit(disp=False)
            forecast =
results.get_forecast(steps=steps)
            forecast_mean =
forecast.predicted_mean.values
            return forecast_mean
        except Exception as e:
            st.write(f"Forecast failed for
{company}: {e}")
            return None

    # Generate CO2 forecasts for all companies
    forecasted_results = []
    for company in
company_grouped_data['Company'].unique():
        forecast_values =
forecast_company_co2(company_grouped_data,
company)
        if forecast_values is not None:
            latest_row = data[(data['Company'] ==
company) &
                              (data['Date'] ==
data[data['Company'] ==
company]['Date'].max())].copy()
            if not latest_row.empty:
                for i in range(6):
                    latest_row[f'Forecasted_CO2_M
onth_{i + 1}'] = forecast_values[i]
                forecasted_results.append(latest_
row)

    # Combine the forecasted results, keeping
only the required rows
    co2_forecast_df =
pd.concat(forecasted_results, ignore_index=True)

    # Only keep the columns necessary for merging
    co2_forecast_df = co2_forecast_df[['Company',
'Date'] + [f'Forecasted_CO2_Month_{i + 1}' for i
in range(6)]]

    st.write("CO2 Emissions Forecast Results:")
    st.write(co2_forecast_df.head())
 # ---------------------------- Step 6: Merge
Results ----------------------------
    st.write("Merging Sustainability Score and
CO2 Forecasts...")

    # Merge CO2 forecasts back into the main
dataset
    combined_data = pd.merge(data,
co2_forecast_df, on=['Company', 'Date'],
how='left')

    st.write("Merged Results:")
    st.write(combined_data.head())

 # ---------------------------- Step 7: Save
Combined Results ----------------------------
    output_file = 'Predictions Result.csv'
    combined_data.to_csv(output_file,
index=False)
    st.success("Your Result saved successfully!")

    # Provide download link
    st.download_button(label="Download
Predictions Result",
data=combined_data.to_csv(index=False),
file_name=output_file)
    st.write("You can now import this file into
Tableau for further analysis!")
```

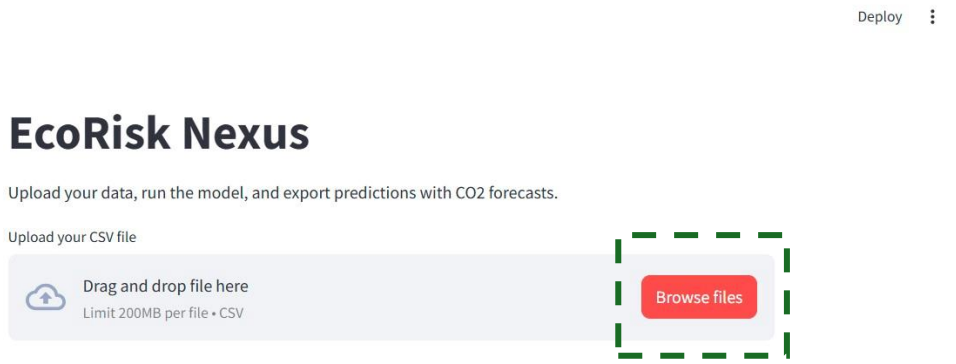
Appendix 8: EcoRisk Nexus Web Application Instructions

1. Click EcoRisk Nexus App on Desktop

2. The browser will open automatically, and this page should appear.



3. Click 'Browse files' or drag and drop your file here. The analysis will start automatically afterward.



4. Click "Download Predictions Result (next page)

Deploy

	Company	Date	Forecasted_CO2_Month_1	Forecasted_CO2_Month_2	Forecasted_CO
0	company_2	2023-10-13 00:00:00	1,331.6882	1,331.6882	
1	company_3	2023-12-05 00:00:00	1,759.2223	1,759.2223	

Merging Sustainability Score and CO2 Forecasts...

Merged Results:

	Company	Project_ID	Date	Incident_Frequency	Downtime_Hours	Energy_Consum
0	company_2	s_6	2023-07-31 00:00:00	1.0113	1.4394	1,186.
1	company_2	s_7	2023-06-03 00:00:00	2.5669	1.3584	1,630.
2	company_2	s_8	2023-05-21 00:00:00	1.9764	4.9844	1,544.
3	company_2	s_4	2023-02-01 00:00:00	1.6164	1.3725	392.
4	company_2	s_5	2023-10-13 00:00:00	0.4213	1.4957	38

Your Result saved successfully!

Download Predictions Result

You can now import this file into Tableau for further analysis!

Appendix 7: Visualization Example Figure from Tableau